PROJECT REPORT : STUDENT DATABASE MANAGEMENT SYSTEM

1. Introduction

This report outlines the design and implementation of a Student Database Management System based on the concepts and syntax provided in the "SQL Syllabus Set.pdf". The project aims to create a functional database to manage student academic records, demonstrating a wide range of SQL skills from basic database creation to advanced querying, data manipulation, and database programming with triggers and stored procedures.

The project will utilize a core student table and a related classroom table to illustrate key database principles like normalization, relationships, and data integrity.

2. Project Objectives

The primary objectives of this project are to:

- Design and implement a relational database for managing student information and marks.
- Demonstrate the use of various SQL command types (DDL, DML, DQL, DCL, TCL).
- Implement data integrity constraints to ensure accurate and reliable data.
- Perform complex data retrieval using filtering, joins, subqueries, and aggregate functions.
- Utilize advanced SQL features like window functions for analytical queries.
- Automate business logic using stored procedures and triggers.

3. Database Design

The database is designed based on the syllabus content.

3.1. Entity-Relationship (E-R) Model

The system consists of two main entities:

- **Student:** Stores core information about each student.
- **Classroom:** Stores classroom assignments for students.

3.2. Relationship

- A **Student** can be assigned to **one or zero** classrooms (as per the sample data).
- A **Classroom** can have **many** students.
- This establishes a **one-to-many** relationship from classroom to student.

3.3. Table Structure (DDL)

The Data Definition Language (DDL) commands to create the database and tables, incorporating constraints, are as follows:

sql

```sql
-- Create the database
CREATE DATABASE mark_list;
USE mark_list;

-- Create the student table with constraints
CREATE TABLE student (
    student_id VARCHAR(10) PRIMARY KEY,  -- PRIMARY KEY (UNIQUE + NOT NULL)
    student_name CHAR(20) NOT NULL,      -- NOT NULL constraint
    tamil INT,
    english INT,
    maths INT,
    total_mark INT,
    percentage INT
);

-- Create the classroom table
CREATE TABLE classroom (
    student_id VARCHAR(10),
    class_name VARCHAR(20),
    -- FOREIGN KEY constraint to maintain referential integrity
    FOREIGN KEY (student_id) REFERENCES student(student_id)
);
```

4. Data Manipulation and Querying (DML & DQL)

This section demonstrates the core operations for managing and retrieving data.

4.1. Inserting Sample Data (DML - INSERT)

sql

```sql
INSERT INTO student (student_id, student_name, tamil, english, maths) VALUES
("AUG001", "ABI", 70, 80, 90),
```

```
("AUG002", "CHRIS", 79, 89, 89),
("AUG008", "Subathara", 89, 45, 67);

INSERT INTO classroom VALUES
("AUG001", "X- A"),
("AUG002", "X- B");
```

4.2. Data Modification (DML - UPDATE)

sql

```
-- Calculate and update total marks and percentage for all students
UPDATE student SET total_mark = tamil + english + maths;
UPDATE student SET percentage = total_mark / 3;
```

4.3. Data Querying (DQL - SELECT with Filtering)

sql

```
-- Filtering with WHERE, AND, OR, NOT, LIKE, IN, BETWEEN
SELECT * FROM student WHERE percentage < 50;
SELECT * FROM student WHERE student_name LIKE 'A%';   -- Starts with A
SELECT * FROM student WHERE student_name IN ('ABI', 'CHRIS');
SELECT * FROM student WHERE tamil BETWEEN 60 AND 80;
```

4.4. Data Querying (DQL - Joins)

sql

```
-- INNER JOIN: Students who are in a classroom
SELECT s.student_id, s.student_name, c.class_name
FROM student s
INNER JOIN classroom c ON s.student_id = c.student_id;

-- LEFT JOIN: All students, with classroom info if available
SELECT s.student_id, s.student_name, c.class_name
FROM student s
LEFT JOIN classroom c ON s.student_id = c.student_id;
```

5. Advanced SQL Concepts

5.1. Aggregate Functions & GROUP BY

Used to perform calculations on sets of rows.

sql

```
-- Find average marks across all students

SELECT AVG(total_mark) AS avg_total_marks FROM student;


-- Count students per class

SELECT c.class_name, COUNT(*) AS total_students

FROM classroom c

GROUP BY c.class_name;
```

5.2. Subqueries

A query nested inside another query. This example finds students with above-average Tamil marks.

sql

```
SELECT student_name, tamil

FROM student

WHERE tamil > (SELECT AVG(tamil) FROM student);
```

5.3. Window Functions

Perform calculations across a set of rows related to the current row without collapsing them.

sql

```
-- RANK students by total marks

SELECT student_id, student_name, total_mark,

    RANK() OVER (ORDER BY total_mark DESC) AS rank_no

FROM student;


-- Calculate a running total of total_mark

SELECT student_id, student_name, total_mark,

    SUM(total_mark) OVER (ORDER BY student_id) AS running_total

FROM student;
```

6. Database Programming

6.1. Stored Procedure with Parameters

A stored procedure that uses IN, OUT, and INOUT parameters to calculate and return a student's total marks and percentage.

sql

```
DELIMITER //


CREATE PROCEDURE calculate_student_percentage(
   IN student_id VARCHAR(10),
   INOUT total_marks INT,
   OUT percentage DECIMAL(5,2)
)
BEGIN
   DECLARE tamil INT;
   DECLARE english INT;
   DECLARE maths INT;

   SELECT tamil, english, maths INTO tamil, english, maths
   FROM student WHERE student.student_id = student_id;

   SET total_marks = tamil + english + maths;
   SET percentage = total_marks / 3.0;
END //


DELIMITER ;
```

6.2. Triggers for Data Integrity and Auditing

- **BEFORE Trigger:** Prevents invalid data from being inserted. This trigger ensures Tamil marks are non-negative.

sql

```
DELIMITER //
```

```
CREATE TRIGGER before_insert_student
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
   IF NEW.tamil < 0 THEN
      SET NEW.tamil = 0;
   END IF;
END //
DELIMITER ;
```

- **AFTER Trigger:** Logs changes to the student table for auditing purposes.

sql

```
DELIMITER //
CREATE TRIGGER after_update_student
AFTER UPDATE ON student
FOR EACH ROW
BEGIN
   INSERT INTO update_log (student_id, old_tamil, new_tamil, update_time)
   VALUES (OLD.student_id, OLD.tamil, NEW.tamil, NOW());
END //
DELIMITER ;
```

7. Conclusion

This project successfully demonstrates the practical application of the SQL concepts outlined in the syllabus. A fully functional Student Database Management System was designed and implemented, showcasing skills in:

- **Database Design:** Creating normalized tables with appropriate constraints.
- **Data Manipulation:** Inserting, updating, and deleting data.
- **Data Querying:** Using complex filtering, joins, aggregate functions, subqueries, and window functions to extract meaningful insights.
- **Database Programming:** Creating stored procedures to encapsulate business logic and triggers to enforce data integrity and automate auditing.